TP D'ÉVALUATION – AGENCE MULTISERVICE

Thème : Gestion Clients et Services

Vous êtes chargé de créer un mini-système pour une **agence multiservice**.
L'agence gère :

- Des clients
- Des services souscrits par les clients

Vous allez pratiquer : **Query Builder, Pagination, Migrations, Seeders et Factories**.

PARTIE 1 — CONFIGURATION DE LA BASE DE DONNÉES

1️⃣ Créer une base dans phpMyAdmin :

agence_multi

2️⃣ Configurer le fichier .env :

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=agence_multi

DB_USERNAME=root

DB_PASSWORD=

3️⃣ Tester la connexion dans le terminal :

php artisan migrate

Si aucune erreur → connexion OK.

PARTIE 2 — MIGRATION CLIENTS

Créer une table clients :

php artisan make:migration create_clients_table

Modifier la migration :

```
public function up()
{
    Schema::create('clients', function (Blueprint $table) {
```

```
        $table->id();
        $table->string('name');
        $table->string('email');
        $table->string('phone');
        $table->timestamps();
    });
}
```

Exécuter la migration :

```
php artisan migrate
```

Vérifier dans phpMyAdmin.

PARTIE 3 — MIGRATION SERVICES

Créer une table services :

```
php artisan make:migration create_services_table
```

Modifier la migration :

```
public function up()
{
    Schema::create('services', function (Blueprint $table) {
        $table->id();
        $table->integer('client_id');
        $table->string('name');
        $table->integer('price');
        $table->timestamps();
    });
}
```

Exécuter la migration :

```
php artisan migrate
```

PARTIE 4 — INSERTION AVEC QUERY BUILDER

Dans routes/web.php, ajouter :

Ajouter un client

```
Route::get('/add-client', function() {
    DB::table('clients')->insert([
        'name' => 'Ali',
        'email' => 'ali@mail.com',
        'phone' => '0600000001'
    ]);
    return "Client ajouté";
});
```

Ajouter un service pour le client 1

```
Route::get('/add-service', function() {
    DB::table('services')->insert([
        'client_id' => 1,
        'name' => 'Nettoyage',
        'price' => 500
    ]);
    return "Service ajouté";
});
```

PARTIE 5 — AFFICHAGE DES DONNÉES

Tous les clients

```
Route::get('/clients', function() {
    $clients = DB::table('clients')->get();
    foreach($clients as $client){
        echo $client->name . " - " . $client->email . "<br>";
    }
});
```

Tous les services

```
Route::get('/services', function() {
    $services = DB::table('services')->get();
```

```
    foreach($services as $service){
        echo $service->name . " - " . $service->price . "<br>";
    }
});
```

PARTIE 6 — MODIFIER ET SUPPRIMER

Modifier le nom d'un client

```
Route::get('/update-client', function() {
    DB::table('clients')->where('id', 1)->update([
        'name' => 'Karim'
    ]);
    return "Client modifié";
});
```

Supprimer un service

```
Route::get('/delete-service', function() {
    DB::table('services')->where('id', 1)->delete();
    return "Service supprimé";
});
```

PARTIE 7 — TRANSACTION SÉCURISÉE

Créer une route /transaction-test pour insérer **client + service** en transaction :

```
Route::get('/transaction-test', function() {
    DB::beginTransaction();
    try {
        $clientId = DB::table('clients')->insertGetId([
            'name' => 'Nadia',
            'email' => 'nadia@mail.com',
            'phone' => '0600000002'
        ]);
```

```
        DB::table('services')->insert([
            'client_id' => $clientId,
            'name' => 'Peinture',
            'price' => 1200
        ]);
        DB::commit();
        return "Transaction réussie";
    } catch (\Exception $e) {
        DB::rollBack();
        return "Erreur, tout annulé";
    }
});
```

PARTIE 8 — PAGINATION

Afficher 5 clients par page dans une vue resources/views/clients.blade.php :

```
Route::get('/clients-paginate', function() {
    $clients = DB::table('clients')->paginate(5);
    return view('clients', compact('clients'));
});
```

Vue clients.blade.php :

```
<h2>Liste des clients</h2>
@foreach($clients as $client)
    {{ $client->name }} - {{ $client->email }}<br>
@endforeach

{{ $clients->links() }}
```

PARTIE 9 — SEEDERS

Créer un seeder

php artisan make:seeder ClientSeeder

Modifier ClientSeeder.php :

```
use Illuminate\Support\Facades\DB;

public function run()
{
    DB::table('clients')->insert([
        'name' => 'Admin',
        'email' => 'admin@agencemulti.com',
        'phone' => '0611111111'
    ]);
}
```

Déclarer le seeder dans DatabaseSeeder.php

```
$this->call(ClientSeeder::class);
```

Exécuter le seeder

php artisan db:seed

PARTIE 10 — FACTORIES

Créer une factory

php artisan make:factory ClientFactory

Modifier ClientFactory.php :

```
public function definition()
{
    return [
        'name' => fake()->name(),
        'email' => fake()->safeEmail(),
        'phone' => '06'.fake()->numberBetween(10000000, 99999999)
    ];
}
```

Utiliser la factory dans DatabaseSeeder.php

\App\Models\Client::factory()->count(20)->create();

Exécuter :

php artisan db:seed

Résultat attendu : 20 clients aléatoires insérés automatiquement dans la base.

QUESTIONS D'ÉVALUATION – QUERY BUILDER & PAGINATION

1. Afficher un client par ID avec Query Builder.
2. Afficher tous les services dont le prix > 1000.
3. Modifier dynamiquement le prix d'un service via l'URL.
4. Supprimer tous les services d'un client donné.
5. Afficher les clients par pages (5 clients/page).
6. Tester une transaction : insérer un client + service.
7. Créer un seeder qui ajoute 2 clients par défaut.
8. Créer une factory et générer 10 clients automatiquement.